



UNIVERSIDAD
DE MÁLAGA



Proyecto Final: Robot de tracción diferencial

Realizado por
Lucía Ramos Gambero

Asignatura
Laboratorio de Robótica

Grado
Ingeniería Electrónica, Robótica y Mecatrónica

En el departamento de
Ingeniería de Sistemas y Automática

Curso 2025/26

Índice general

1	Introducción y descripción del proyecto	3
2	Montaje del robot	4
2.1.	Procedimiento y conexionado	4
3	Tarea 1: línea recta 20 seg a 0.1 m/s.	7
3.1.	Objetivo	7
3.2.	Procedimiento	7
3.2.1.	Constantes físicas	7
3.2.2.	Modelo Cinemático Inverso	7
3.2.3.	Estrategia de control en bucle abierto	8
3.2.4.	Temporización y Lógica de Estado	8
4	Tarea 2: Recorrer 2 metros en línea recta.	9
4.1.	Objetivo	9
4.2.	Procedimiento	9
4.2.1.	Odometría y Cálculo de Distancia	9
4.2.2.	Control de Avance	10
4.2.3.	Distancia recorrida y estrategia de parada	10
5	Tarea 3: Trazar una circunferencia	11
5.1.	Objetivo	11
5.2.	Procedimiento	11
5.2.1.	Cálculo de velocidades	11
5.2.2.	Cinemática Inversa	11
5.2.3.	Control en Bucle Abierto y Calibración	12
5.2.4.	Temporización	12
6	Tarea 4: Recorrer 2 metros en línea recta con control de orientación.	13
6.1.	Objetivo	13
6.2.	Procedimiento	13
6.2.1.	Control de Orientación	13
6.2.2.	Control PID de Velocidad	13
6.2.3.	Odometría y Condición de Parada	14
7	Tarea 5: Seguimiento de trayectoria (Follow the carrot)	15
7.1.	Objetivo	15
7.2.	Procedimiento	15
7.2.1.	Odometría	15
7.2.2.	Algoritmo de navegación	15

7.2.3. Control de velocidad	16
8 Tarea 6: Sensores de proximidad	17
8.1. Objetivo	17
8.2. Procedimiento	17
8.2.1. Lectura de sensores	17
8.2.2. Lógica de control y seguridad	17
8.2.3. Cinemática Inversa y PID	19
9 Tarea 7	20
9.1. ESP32 micro-ROS	20
9.1.1. Configuración y Comunicación	20
9.1.2. Bucle de Control Principal (Timer Callback)	20
9.1.3. Interfaz ROS 2	21
10 Tarea 10: Tarea Opcional- Control autónomo desde móvil . .	22
10.0.1. Infraestructura de Comunicación	22
10.0.2. Lógica de Control y Seguridad	22
10.0.3. Resumen de Flujo	23

1. Introducción y descripción del proyecto

El presente documento recoge el desarrollo del proyecto final de la asignatura *Laboratorio de Robótica*. El objetivo del proyecto es realizar un robot de tracción diferencial. Para ello, se realizan diferentes tareas, aumentando gradualmente la dificultad de las mismas:

- **Tarea 1:** Movimiento en línea recta con referencia de velocidad de 0.1 m/s durante 20 segundos.
- **Tarea 2:** Recorrer 2 metros en línea recta.
- **Tarea 3:** Trazar una circunferencia de 0.4 metros de radio y una velocidad angular $w = \frac{2\pi}{10}$ rad/s durante 10 segundos.
- **Tarea 4:** Recorrer 2 metros en línea recta con control de orientación.
- **Tarea 5:** Implementar un algoritmo de seguimiento de trayectoria (Follow the carrot).
- **Tarea 6:** Añadir sensores de proximidad, si detecta un obstáculo el vehículo se para, si desaparece el obstáculo el vehículo sigue con su tarea.
- **Tarea 7:** Implementar al menos la tarea 5 con micro-ROS:
 - Comunicación mediante WiFi.
 - El ESP32 debe suscribirse al topic `cmd_vel`.
 - El ESP32 debe publicar su posición (x, y, θ) .

2. Montaje del robot

2.1. Procedimiento y conexionado

Se detallan los pasos seguidos para realizar correctamente las conexiones entre todos los elementos, garantizando el correcto funcionamiento. Para llevarlas a cabo se tiene en cuenta el esquema de conexiones del motor, proporcionado por el fabricante.

- **Conexión del microcontrolador con el driver L298N:**

Las entradas IN1 e IN2 del driver se conectan a los pines 32 y 33 del ESP32, respectivamente. Las entradas IN3 e IN4, se conectan a los pines 27 y 24. A través de estos pines, el microcontrolador envía las señales PWM de entrada, y el driver actúa como una etapa de potencia, que convierte estas señales de bajo nivel en señales capaces de suministrar la corriente y tensión necesarias a los motores.

- **Conexión del driver con los motores:**

Se conectan las salidas de potencia OUT1 y OUT2 del driver con las entradas del motor 1 (cable rojo y blanco respectivamente). Para el motor 2, se conectan sus entradas a las salidas OUT3 y OUT4. Estas salidas son las encargadas de suministrar al motor con la corriente necesaria. De nuevo, se muestra como el driver actúa como etapa intermedia entre elementos que requieren de diferentes niveles de tensión y corriente.

- **Conexión de las señales de los encoders:**

El encoder acoplado al eje del motor genera dos señales en cuadratura (A y B), que se utilizan para contar los pulsos generados durante la rotación del eje del motor. Por ello, cada encoder debe ir conectado a las entradas digitales del microprocesador configuradas con las interrupciones que llevan a cabo esta cuenta de pulsos. Por ello, para el primer motor, se conecta el cable amarillo del encoder con el pin 25 del ESP32, y el cable verde con el pin 26. Para el segundo, se conecta el cable amarillo con el pin 4, y el cable verde con el PIN 13.

- **Alimentación del sistema:**

El driver necesita una alimentación para poder suministrar la corriente adecuada al motor de corriente continua. Se conecta el terminal positivo UPS+ a los 12V del driver, y el terminal negativo UPS- a la masa común del ESP32.

- **Referencia común de GND:**

Es fundamental que todos los elementos compartan tierra común para que las señales y lecturas tengan un punto referencia de voltaje y de esta forma, sean coherentes entre sí. Por ello se conecta tanto el GND del ESP32 y de los

2.1. PROCEDIMIENTO Y CONEXIONADO



UNIVERSIDAD
DE MÁLAGA



encoders con GND del driver, además de la de la fuente de alimentación ya comentada en el punto anterior.

Finalmente, las conexiones se resumen según las tablas 2.1 y 2.2.

	Motor 1		Motor 2	
	IN1	PIN 32	IN3	PIN 27
Driver-ESP32	IN2	PIN 33	IN4	PIN 14
	OUT1	Blanco	OUT3	Rojo
Driver-Motor	OUT2	Rojo	OUT4	Blanco
	Amarillo	PIN 26	Amarillo	PIN 4
Encoder-ESP32	Verde	PIN 25	Verde	PIN 13

Tabla 2.1: Conexiones del sistema por motor

Alimentación	UPS+ alimentación	12 V driver
	UPS- alimentación	GND ESP32
Tierra común	GND ESP32 + GND encoders	GND drivers

Tabla 2.2: Conexiones comunes

Además se incluye un interruptor entre la conexión UPS+ del portapilas y los 12 V del driver.

	Izquierda	Centro	Derecha
Out (Echo)	PIN 39	PIN 35	PIN 34

Tabla 2.3: Asignación de pines del ESP32 según el sensor

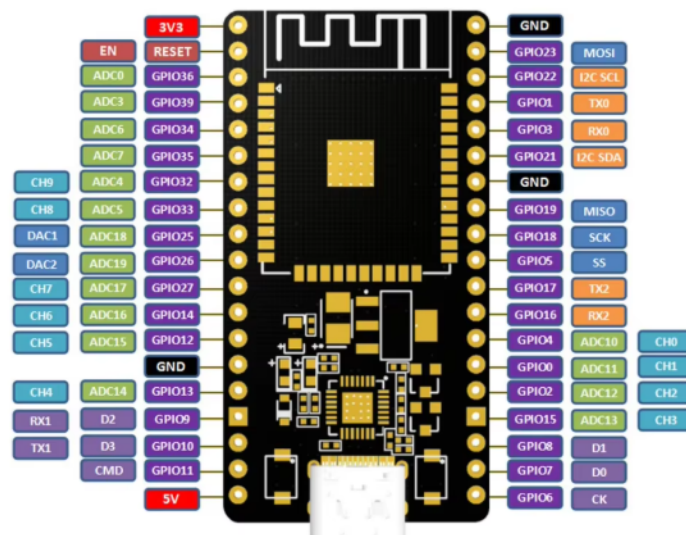


Figura 2.1: Pines ESP32

3. Tarea 1: línea recta 20 seg a 0.1 m/s.

3.1. Objetivo

El objetivo de esta tarea es realizar un movimiento en línea recta con referencia de velocidad de 0.1 m/s durante 20 segundos, sin control de corrección, es decir, un movimiento con control en bucle abierto. Para ello, se aplica una señal PWM fija a los motores para observar la respuesta física del sistema en ausencia de control de corrección.

3.2. Procedimiento

3.2.1. Constantes físicas

Para realizar una traducción de los datos que recopilan los encoders (cuentas por vuelta de cada eje) a magnitudes físicas reales, se definen dos constantes:

- **Radio de la rueda** $R = 0,0325m$
- **Distancia entre ruedas** $L = 0,197m$
- **Cuentas por vuelta** $N = 1680$

```
const float R = 0.0325;  
const float L = 0.197;  
const float N_VUELTA = 1680.0;
```

3.2.2. Modelo Cinemático Inverso

Para conseguir el movimiento deseado, es necesario transformar las velocidades lineal (v) y angular (w) del robot en las velocidades lineales individuales de cada rueda. Dado que se requiere una línea recta, la velocidad angular de referencia es $w = 0$ durante todo el recorrido. Se aplican las ecuaciones de la cinemática inversa para un robot diferencial:

$$v_{izq} = v - \frac{w \cdot L}{2}$$
$$v_{der} = v + \frac{w \cdot L}{2}$$

```
double v_izq = comando_lineal_v - (comando_angular_w * L / 2.0);  
double v_der = comando_lineal_v + (comando_angular_w * L / 2.0);
```

3.2. PROCEDIMIENTO

3.2.3. Estrategia de control en bucle abierto

Al tratarse de un control en bucle abierto, no se utiliza la realimentación de los encoders para corregir el error en tiempo real. En su lugar, se emplea una constante de calibración (K_{OPEN_LOOP}). Esta constante se ha obtenido experimentalmente, estableciendo el duty de la señal PWM con la velocidad buscada (0.1 m/s). El microcontrolador envía la señal al driver, que envía la tensión correspondiente a los motores. En este caso los encoders solo sirven para medir, no para controlar.

Esta constante se ha ajustado para conseguir que se corresponda a una buena respuesta para la velocidad de 0.1 (m/s) .

```
const float K_OPEN_LOOP = 2200.0;
// ...
// Mapeo directo a PWM usando el factor de calibracion
double pwm_L = v_izq * K_OPEN_LOOP;
double pwm_R = v_der * K_OPEN_LOOP;

setMotor(PULSO_L_A, PULSO_L_B, pwm_L);
setMotor(PULSO_R_A, PULSO_R_B, pwm_R);
```

3.2.4. Temporización y Lógica de Estado

Para garantizar la duración exacta de la tarea, se utiliza la función `millis()` para controlar el tiempo de ejecución. El robot inicia el movimiento estableciendo las referencias y, una vez transcurridos 20000 ms (20 segundos), se envían señales a los motores para que paren ($\text{PWM} = 0$) y se desactiva la bandera de control.

```
// --- TAREA 1: RECTA 20 SEGUNDOS ---
if (estado_recien_iniciado) {
    Serial.println("INICIANDO TAREA 1: 0.1 m/s por 20 segundos");
    tiempo_inicio_estado = millis();

    // Introducimos la velocidad objetivo fisica
    comando_lineal_v = 0.1;
    comando_angular_w = 0.0;

    estado_recien_iniciado = false;
}

// --- COMPROBAR TIEMPO (20 SEGUNDOS) ---
// Matematicamente: 0.1 m/s * 20 s = 2.0 metros
if (millis() - tiempo_inicio_estado > 20000) {
    setMotor(PULSO_L_A, PULSO_L_B, 0);
    setMotor(PULSO_R_A, PULSO_R_B, 0);
    robot_activo = false;
    return;
}
```

4. Tarea 2: Recorrer 2 metros en línea recta.

4.1. Objetivo

Para esta tarea, se busca que el robot recorra 2 metros en línea recta. Aunque pueda parecer la misma tarea que la anterior, que se basaba en ir a una velocidad estimada y parar en un tiempo determinado, en esta el tiempo no es lo importante, si no que en este caso interesa medir la distancia exacta en tiempo real, usando la odometría, y que la condición de parada se defina según la distancia y no el tiempo.

4.2. Procedimiento

4.2.1. Odometría y Cálculo de Distancia

Para medir la distancia recorrida, se utilizan las interrupciones de los encoders para contar los pulsos de cada rueda (*counts_L* y *counts_R*). En cada ciclo de control ($T_s = 10ms$), se calcula la variación de pulsos y se transforma a distancia lineal recorrida (*ds*) utilizando el radio de la rueda (*R*) y la resolución del encoder (*N_VUELTA*).

La posición longitudinal *x* se actualiza integrando estos desplazamientos diferenciales:

$$\Delta s = \frac{2\pi R \cdot \Delta counts}{N_{VUELTA}}$$
$$x_{pos}(k) = x_{pos}(k-1) + \frac{\Delta s_L + \Delta s_R}{2}$$

```
// ODOMETRIA
long dL = counts_L - odom_prev_L;
long dR = counts_R - odom_prev_R;

double dsL = (2.0 * PI * R * dL) / N_VUELTA;
double dsR = (2.0 * PI * R * dR) / N_VUELTA;
double ds = (dsR + dsL) / 2.0;

// Actualizacion de la posicion longitudinal
x_pos += ds;
```

4.2. PROCEDIMIENTO

4.2.2. Control de Avance

Durante el trayecto, el robot avanza con una consigna de velocidad constante de $0,2 \text{ m/s}$. Al igual que en la tarea anterior, se utiliza un factor de conversión ($K_{CONVERSION}$) para traducir esta velocidad a una señal PWM.

```
// Velocidad aproximada durante el trayecto
double comando_lineal_v = 0.2;
double pwm = comando_lineal_v * K_CONVERSION;

setMotor(PULSO_L_A, PULSO_L_B, pwm);
setMotor(PULSO_R_A, PULSO_R_B, pwm);
```

4.2.3. Distancia recorrida y estrategia de parada

La distancia exacta se calcula como el valor absoluto de la diferencia entre la posición actual en la que se encuentra el robot y la distancia inicial.

En esta tarea, se observa como el robot desliza unos centímetros después de llegar a la distancia de 2 metros. Esto se debe a que es difícil controlar la inercia del vehículo en un sistema en lazo abierto.

Para solucionar esto y lograr una parada precisa a los 2 metros, se han implementado dos estrategias en el código:

1. **Compensación de distancia:** Se establece el umbral de parada en $1,75\text{m}$ en lugar de $2,0\text{m}$, ya que experimentalmente se ha determinado que el robot desliza aproximadamente 20 cm debido a la inercia a la velocidad de $0,2 \text{ m/s}$.
2. **Frenado Activo (Contramarcha):** Al alcanzar el umbral, en lugar de simplemente poner el PWM a 0, se invierte la polaridad de los motores con una señal negativa (PWM -400) durante un periodo muy breve (50 ms). Esto se hace para forzar una frenada a tiempo, aunque un poco brusca.

```
// Compensacion de distancia
if (distancia_recorrida >= 1.75) {

// Frenado activo
setMotor(PULSO_L_A, PULSO_L_B, -400);
setMotor(PULSO_R_A, PULSO_R_B, -400);
delay(50);

// Apagado final
setMotor(PULSO_L_A, PULSO_L_B, 0);
setMotor(PULSO_R_A, PULSO_R_B, 0);

robot_activo = false;
return;
}
```

5. Tarea 3: Trazar una circunferencia

5.1. Objetivo

El objetivo de esta tarea es que el robot realice una circunferencia con un radio de $0,4\text{ m}$ durante 10 segundos, a una velocidad angular de $\omega = \frac{2\pi}{10}\text{ rad/s}$.

5.2. Procedimiento

5.2.1. Cálculo de velocidades

Para generar la circunferencia, se calculan en primer lugar las velocidades tanto lineal como angular. La velocidad angular ya es definida en el enunciado de la tarea, mientras que la lineal se calcula a partir de la relación del movimiento circular uniforme.

- **Velocidad Angular (ω):** .

$$\omega_{ref} = \frac{2\pi}{10}\text{ rad/s}$$

- **Velocidad Lineal (v):**

$$v_{ref} = \omega_{ref} \cdot R_{giro}$$

Se calcula para un radio $R_{giro} = 0,4\text{ m}$.

```
// w_ref: Velocidad para dar una vuelta completa (2*PI) en 10 segundos
comando_angular_w = (2.0 * PI) / 10.0;
// v_ref: Define el radio del circulo (w * radio)
comando_lineal_v = comando_angular_w * 0.4; // Radio de 0.4 metros
```

5.2.2. Cinemática Inversa

A diferencia del movimiento en línea recta, para describir una curva, las ruedas del robot deben girar a velocidades diferentes. Se aplican las ecuaciones de cinemática diferencial, teniendo en cuenta la distancia entre ruedas (L):

$$v_{der} = v_{ref} + \frac{\omega_{ref} \cdot L}{2}$$
$$v_{izq} = v_{ref} - \frac{\omega_{ref} \cdot L}{2}$$

```
// --- CINEMATICA INVERSA ---  
// Calculamos la velocidad diferencial para que el robot gire  
double v_der = comando_lineal_v + (comando_angular_w * L / 2.0);  
double v_izq = comando_lineal_v - (comando_angular_w * L / 2.0);
```

5.2.3. Control en Bucle Abierto y Calibración

Se mantiene la estrategia de control descrita anteriormente, ajustando la constante definida. Este ligero aumento respecto a la tarea de línea recta es necesario para compensar el incremento de fricción que se produce en las ruedas durante el deslizamiento lateral.

```
// Traducimos la velocidad fisica (m/s) a potencia PWM  
double pwm_L = v_izq * K_CONVERSION;  
double pwm_R = v_der * K_CONVERSION;  
  
setMotor(PULSO_L_A, PULSO_L_B, pwm_L);  
setMotor(PULSO_R_A, PULSO_R_B, pwm_R);
```

5.2.4. Temporización

El control de la duración se realiza mediante la función `millis()`. Una vez transcurridos los 10 segundos exactos, se detienen los motores para finalizar la circunferencia.

```
// Comprobar si han pasado los 10 segundos  
if (millis() - tiempo_inicio_estado > 10000) {  
    setMotor(PULSO_L_A, PULSO_L_B, 0);  
    setMotor(PULSO_R_A, PULSO_R_B, 0);  
    robot_activo = false;  
    return;  
}
```

Esto último funciona de la misma forma que en la tarea 2.

6. Tarea 4: Recorrer 2 metros en línea recta con control de orientación.

6.1. Objetivo

El objetivo es recorrer la misma distancia de 2 metros que en la tarea 2, pero esta vez garantizando que el robot mantenga una trayectoria recta, sin desviaciones. Para ello, se abandona el control en lazo abierto y se implementa un sistema de control en lazo cerrado que corrige tanto la velocidad de las ruedas (mediante PIDs) como el error de orientación (mediante un controlador proporcional sobre θ).

6.2. Procedimiento

6.2.1. Control de Orientación

Para asegurar que el robot no se desvíe, se almacena la orientación inicial (θ_{ref}) al comenzar el movimiento. En cada ciclo, se calcula el error de orientación y se genera una velocidad angular de corrección (w_{ref}) proporcional a dicho error.

$$e_{\theta} = \theta_{ref} - \theta_{actual}$$

$$\omega_{ref} = K_{p_orientacion} \cdot e_{\theta}$$

En el código, se ha ajustado una ganancia $K_{p_orientacion} = 2,2$ y se utiliza una función de normalización para evitar saltos discontinuos en los ángulos (por ejemplo, de π a $-\pi$).

```
float v_ref = 0.20;
// Controlador Proporcional de Orientacion
float w_ref = Kp_orientacion * normalizeAngle(theta_ref - theta);

// Mezcla de velocidades (Cinematica Inversa)
double v_wheel_R = v_ref + w_ref * (L / 2.0);
double v_wheel_L = v_ref - w_ref * (L / 2.0);
```

6.2.2. Control PID de Velocidad

Una vez obtenidas las velocidades lineales requeridas para cada rueda (v_{wheel_L}, v_{wheel_R}), estas se convierten a cuentas de encoder por segundo para establecer la referencia de los controladores PID de bajo nivel. Se utilizan los

6.2. PROCEDIMIENTO

parámetros ajustados en la práctica ($K_p = 0,17, K_i = 2,0, K_d = 0,005$) para garantizar que las ruedas sigan la referencia.

```
// Conversion a Cuentas/s
double ref_L = (v_wheel_L / (2.0 * PI * R)) * N_VUELTA;
double ref_R = (v_wheel_R / (2.0 * PI * R)) * N_VUELTA;

// Calculo de velocidad real
double vel_real_L = (counts_L - prev_counts_L) / Ts;
double vel_real_R = (counts_R - prev_counts_R) / Ts;

// Lazo de control PID
double pwm_L = computePID(ref_L, vel_real_L, err_sum_L, last_err_L, der_filt_L);
double pwm_R = computePID(ref_R, vel_real_R, err_sum_R, last_err_R, der_filt_R);
```

6.2.3. Odometría y Condición de Parada

La posición se actualiza integrando el desplazamiento lineal y angular en cada ciclo. La condición de parada sigue basada en la distancia euclídea desde el punto de inicio. Se realiza una parada más suave.

```
double distancia = sqrt(pow(x_pos - start_x, 2) + pow(y_pos - start_y, 2));

// --- AJUSTE TAREA 4: PARADA Y FRENADO SUAVE ---
if (distancia >= 1.75) {
    // FRENADO MAS SUAVE (Menos potencia y menos tiempo)
    setMotor(PULSO_L_A, PULSO_L_B, -350);
    setMotor(PULSO_R_A, PULSO_R_B, -350);
    delay(40);

    setMotor(PULSO_L_A, PULSO_L_B, 0);
    setMotor(PULSO_R_A, PULSO_R_B, 0);
    robot_activo = false;
    return;
}
```

7. Tarea 5: Seguimiento de trayectoria (Follow the carrot)

7.1. Objetivo

Para esta tarea, se busca implementar un algoritmo de seguimiento de trayectoria (Follow the carrot). El objetivo es recorrer una trayectoria que define un cuadrado de 0.5m x 0.5m. Para ello se utiliza un control en bucle cerrado, con controlador PID y estimación de posición mediante odometría.

7.2. Procedimiento

7.2.1. Odometría

Para estimar su posición, se calcula en cada ciclo de control (T) el desplazamiento realizado, según el número de pulsos por vuelta, $N_VUELTAS$. La pose se calcula actualizando las coordenadas y la orientación haciendo uso de las ecuaciones de cinemática diferencial. Además, se utiliza una función auxiliar `normalizeAngle` para que el ángulo se encuentre siempre entre π y $-\pi$.

```
// --- ODOMETRIA ---
long d_counts_L = counts_L - prev_counts_L;
long d_counts_R = counts_R - prev_counts_R;
prev_counts_L = counts_L;
prev_counts_R = counts_R;

double ds_L = (2.0 * PI * R * d_counts_L) / N_VUELTA;
double ds_R = (2.0 * PI * R * d_counts_R) / N_VUELTA;
double ds = (ds_R + ds_L) / 2.0;
double dtheta = (ds_R - ds_L) / L;

x_pos += ds * cos(theta + dtheta / 2.0);
y_pos += ds * sin(theta + dtheta / 2.0);
theta += dtheta;
theta = normalizeAngle(theta);
```

7.2.2. Algoritmo de navegación

El robot persigue una serie de coordenadas objetivo, definido en el array `pathX` y `pathY`. En este caso, la trayectoria que debe seguir es un cuadrado de lado 0.5m.

```
float pathX[] = {0.0, 0.5, 0.5, 0.0, 0.0};  
float pathY[] = {0.0, 0.0, 0.5, 0.5, 0.0};
```

La lógica se divide en dos estados, en función del error de ángulo:

1. Giro: Si el error de orientación es mayor a 0.2 rad, el robot se detiene y gira sobre su eje 90°.

```
if (abs(angle_error) > 0.2) { // > ~11 grados  
    v_ref = 0.0;  
    // Giro proporcional (puedes subir este 3.0 si gira lento)  
    w_ref = 3.0 * angle_error;  
  
    // Saturar giro  
    if (w_ref > 2.0) w_ref = 2.0;  
    if (w_ref < -2.0) w_ref = -2.0;  
}
```

2. Avance con corrección: Avanza a velocidad constante aplicando el PID implementado, y conseguir mantenerse en su trayectoria fija hacia el objetivo.

```
else {  
    // AVANZAR Y CORREGIR SUAVEMENTE  
    v_ref = 0.20;  
    w_ref = 1.5 * angle_error;  
}
```

3. Tolerancia al llegar: Si la distancia al punto objetivo es menor a 5 cm, se salta al siguiente punto objetivo.

```
if (dist_error < 0.05) { // Llegada al punto (5cm tolerancia)  
    currentPoint++;  
    // Resetear integrales para evitar tirones  
    err_sum_L = 0; err_sum_R = 0;  
    // Parada breve  
    setMotor(PULSO_L_A, PULSO_L_B, 0);  
    setMotor(PULSO_R_A, PULSO_R_B, 0);  
    return;  
}
```

7.2.3. Control de velocidad

Para el control de velocidad se implementa el PID con filtro derivativo de primer orden, además de anti-windup. De esta forma se evitan picos de corriente en los motores y saturación en el sistema ante cambios bruscos.

8. Tarea 6: Sensores de proximidad

8.1. Objetivo

Para esta tarea, se añaden sensores de proximidad. Si detecta un obstáculo el vehículo se para, si desaparece el obstáculo el vehículo sigue con su tarea.

8.2. Procedimiento

8.2.1. Lectura de sensores

Se utilizan tres sensores de proximidad conectados a las entradas analógicas del ESP32. La detección se basa en la lectura del nivel de tensión proporcionado por el sensor, el cual varía proporcionalmente a la distancia. Se establece un umbral de 22 cm para determinar si el objeto está demasiado cerca.

```
// --- PINES SENSORES (Entrada Analogica) ---
const int PIN_SENSOR_1 = 34; // Derecha
const int PIN_SENSOR_2 = 35; // Izquierda
const int PIN_SENSOR_3 = 39; // Centro
const int UMBRAL_OBSTACULO = 300; // Valor de calibracion (cerca)

// Funcion de deteccion
bool hay_obstaculo() {
    int val1 = analogRead(PIN_SENSOR_1);
    int val2 = analogRead(PIN_SENSOR_2);
    int val3 = analogRead(PIN_SENSOR_3);

    // Si la lectura es menor al umbral, implica distancia critica
    if (val1 < UMBRAL_OBSTACULO || val2 < UMBRAL_OBSTACULO || val3 <
        UMBRAL_OBSTACULO) {
        return true;
    }
    return false;
}
```

8.2.2. Lógica de control y seguridad

El flujo de la lógica principal del bucle de control queda de la siguiente forma:

1. **Detección y Parada:** Si la función `hay_obstaculo()` devuelve `true`, se ignoran los comandos de movimiento y se fuerza la velocidad lineal (v) y angular (w) a 0. Se activa una bandera de estado `en_pausa_obstaculo`.

```
if (obstaculo_presente) {  
    if (!en_pausa_obstaculo) {  
        // Deteccion inicial  
        en_pausa_obstaculo = true;  
        esperando_reinicio = false;  
    }  
    // Parada  
    comando_lineal_v = 0.0;  
    comando_angular_w = 0.0;  
}
```

2. **Espera de seguridad:** Cuando el obstáculo desaparece, el robot no arranca inmediatamente. Se inicia un temporizador de 3 segundos (`millis() - tiempo_ultimo_obstaculo`) para garantizar que el entorno es seguro antes de reanudar la marcha.

```
else if (en_pausa_obstaculo && !esperando_reinicio) {  
    // El obstaculo acaba de desaparecer  
    tiempo_ultimo_obstaculo = millis();  
    esperando_reinicio = true;  
}  
  
if (esperando_reinicio) {  
    // Mantener parada durante 3 segundos  
    comando_lineal_v = 0.0;  
    if (millis() - tiempo_ultimo_obstaculo > 3000) {  
        esperando_reinicio = false;  
        en_pausa_obstaculo = false; // Reanudar  
    }  
}
```

3. **Navegación normal:** Si no hay obstáculos y no se está en tiempo de espera, el robot ejecuta su consigna de velocidad por defecto (en este caso, avance en línea recta).

```
else {  
    // ESTADO NORMAL: Avance  
    comando_lineal_v = 0.2;  
    comando_angular_w = 0.0;  
}
```

8.2. PROCEDIMIENTO

8.2.3. Cinemática Inversa y PID

Finalmente, las velocidades deseadas (v y w) calculadas por la lógica de seguridad se transforman a velocidades de rueda y se regulan mediante el PID ajustado previamente, asegurando un movimiento suave tanto en el frenado de emergencia como en la reanudación.

```
// Conversion a ticks por segundo para los controladores PID
double v_der = comando_lineal_v + (comando_angular_w * DISTANCIA_ENTRE_RUEDAS /
    2.0);
double v_izq = comando_lineal_v - (comando_angular_w * DISTANCIA_ENTRE_RUEDAS /
    2.0);

objetivo_vel_motor1 = (v_der / (2.0 * PI * RADIO_RUEDA)) * TICKS_POR_VUELTA;
objetivo_vel_motor2 = (v_izq / (2.0 * PI * RADIO_RUEDA)) * TICKS_POR_VUELTA;
```

9. Tarea 7

Se implementa el uso de microRos, con los topics `cmd_vel` y `turtlebot_pose`.

```
rclcpp_subscription_init_default(
    &subscriber, &node, ROSIDL_GET_MSG_TYPE_SUPPORT(geometry_msgs, msg, Twist),
    "cmd_vel");

rclcpp_publisher_init_default(
    &publisher, &node, ROSIDL_GET_MSG_TYPE_SUPPORT(geometry_msgs, msg, Pose2D),
    "turtlebot_pose");
```

En cuanto a la lógica, esta es análoga a la explicada anteriormente.

9.1. ESP32 micro-ROS

El código implementa un nodo completo de ROS 2 en el microcontrolador ESP32 utilizando la librería `micro_ros_arduino`.

9.1.1. Configuración y Comunicación

El sistema se inicia estableciendo una conexión WiFi (SSID) y conectándose mediante protocolo UDP al **micro-ROS Agent** ubicado en la IP 172.20.10.4. Esto permite al ESP32 actuar como un nodo de la red ROS 2 sin necesidad de un sistema operativo completo.

9.1.2. Bucle de Control Principal (Timer Callback)

La función `timer_callback` es el núcleo del sistema. Se ejecuta estrictamente cada $T_s = 10\text{ms}$ (100 Hz) y realiza cuatro tareas secuenciales:

1. Odometría (Cálculo de Posición)

Se calcula el desplazamiento del robot basándose en la diferencia de cuentas de los encoders (Δcounts).

$$ds = \frac{ds_R + ds_L}{2}, \quad d\theta = \frac{ds_R - ds_L}{L} \quad (9.1)$$

Donde L es la distancia entre ruedas. La posición global (x, y, θ) se actualiza mediante integración aproximada de Runge-Kutta de orden inferior:

$$x_{k+1} = x_k + ds \cdot \cos(\theta_k + d\theta/2) \quad (9.2)$$

2. Publicación de Estado

El nodo publica la posición estimada en el topic `/turtlebot_pose` utilizando un mensaje de tipo `geometry_msgs/Pose2D`.

3. Cinemática Inversa

Convierte las velocidades deseadas recibidas de ROS (v, ω) en velocidades angulares para cada rueda.

4. Control PID de Velocidad

Para garantizar que los motores giren a la velocidad deseada a pesar de la fricción o carga, se implementa un controlador PID discreto para cada rueda.

9.1.3. Interfaz ROS 2

El nodo define la siguiente interfaz de comunicación:

- **Suscriptor (Input):** Topic `/cmd_vel` (tipo `Twist`). Recibe comandos de velocidad lineal y angular.
- **Publicador (Output):** Topic `/turtlebot_pose` (tipo `Pose2D`). Envía la telemetría de odometría.

10. Tarea 10: Tarea Opcional- Control autónomo desde móvil

Este código implementa un sistema de control autónomo en el ESP32, que actúa simultáneamente como Punto de Acceso WiFi, Servidor Web y controlador de los actuadores. A diferencia de la arquitectura micro-ROS, este sistema no depende de un PC externo para el procesamiento.

10.0.1. Infraestructura de Comunicación

El microcontrolador configura una red WiFi local con el SSID "labrob".

- La página web envía peticiones HTTP GET a endpoints específicos (/F, /B, /L, /R, /S) cuando el usuario presiona los botones.
- Estas peticiones actualizan las referencias de velocidad global (v_{ref}, ω_{ref}) sin necesidad de recargar la página.

10.0.2. Lógica de Control y Seguridad

1. Evasión de Obstáculos Reactiva

Se leen tres sensores infrarrojos analógicos (pines 34, 35, 39). Si la lectura cae por debajo del umbral (300), se detecta un obstáculo. Por seguridad, si el robot avanza ($v > 0$) y detecta un obstáculo, la velocidad objetivo se fuerza a 0 inmediatamente.

2. Suavizado de Movimiento (Rampa)

Para evitar picos de corriente y movimientos bruscos, las referencias de velocidad no se aplican directamente, sino que pasan por un filtro de suavizado exponencial:

$$v_{actual} = v_{actual} \cdot (1 - \alpha) + v_{target} \cdot \alpha \quad (10.1)$$

Donde $\alpha = 0,05$ es el factor de rampa. Esto genera una aceleración y desaceleración progresiva.

3. Control PID

Al igual que en versiones anteriores, se calcula la cinemática inversa para obtener las velocidades requeridas en cada rueda y se aplica un control PID independiente.

10.0.3. Resumen de Flujo

El loop principal del programa alterna entre dos tareas:

1. `server.handleClient()`: Atiende las peticiones WiFi de los usuarios.
2. `ejecutarControl()`: Se ejecuta solo cuando el Timer lo indica (100 veces por segundo) para recalcular sensores y actuadores.

La web se ve según la figura 10.1.



Figura 10.1: Interfaz gráfica